

IPA ASSIGNMENT - 21

Name - Nishant Nahar

Roll Number - 241551078

1. LZW Coding Technique

```
from PIL import Image
import numpy as np
def lzw_encode(data):
    dictionary = {bytes([i]): i for i in range(256)}
    next_code = 256
    w = bytes([data[0]]) compressed = []
    for pixel in data[1:]: c = bytes([pixel])
        wc = w + c if wc in dictionary:
            w = wc else:
                compressed.append(dictionary[w])
                dictionary[wc] = next_code
                next_code += 1 w = c
    if w: compressed.append(dictionary[w]) return compressed
def lzw_decode(compressed):
    dictionary = {i: bytes([i]) for i in range(256)} next_code = 256
    w = dictionary[compressed[0]]
    output = bytearray(w)
    for code in compressed[1:]:
        if code in dictionary:
            entry = dictionary[code]
        elif code == next_code:
            entry = w + w[:1] else:
                raise ValueError("Invalid LZW code")
        output.extend(entry)
        dictionary[next_code] = w + entry[:1]
        next_code += 1
        w = entry
    return np.frombuffer(output, dtype=np.uint8)
image = Image.open("img_2.jpg").convert("L")
img_array = np.array(image)
pixels = img_array.flatten()
print("Original Image Size:", img_array.shape)
print("Number of Pixels:", len(pixels))
compressed = lzw_encode(pixels)
print("Number of LZW Codes:", len(compressed))
decoded_pixels = lzw_decode(compressed)
decoded_image = decoded_pixels.reshape(img_array.shape)
Image.fromarray(decoded_image).save("reconstructed.png")
print("LZW Encoding and Decoding Completed.")
print("Reconstructed image saved as reconstructed.png")
original_bits = len(pixels) * 8
max_code = max(compressed)
bits_per_code = max(1, max_code.bit_length())
compressed_bits = len(compressed) * bits_per_code
```

```
compression_ratio = original_bits / compressed_bits
print("\nOriginal bits:", original_bits)
print("Compressed bits:", compressed_bits)
print("Compression Ratio:", compression_ratio)
```

OUTPUT



2. RLC Coding

```
from PIL import Image
import numpy as np
def rlc_encode(pixels):
    encoded = []
    current = pixels[0]
    count = 1
    for pixel in pixels[1:]:
        if pixel == current: count += 1
        else: encoded.append((int(current), count))
            current = pixel
            count = 1
    encoded.append((int(current), count))
    return encoded
def rlc_decode(encoded):
    decoded = []
    for pixel, count in encoded:
        decoded.extend([pixel] * count)
    return np.array(decoded, dtype=np.uint8)
image = Image.open("img_2.jpg").convert("L")
img_array = np.array(image)
pixels = img_array.flatten()
print("Original image shape:", img_array.shape)
print("Original number of pixels:", len(pixels))
encoded = rlc_encode(pixels)
print("Number of RLC pairs:", len(encoded))
print("\nFirst 20 RLC pairs:")
print(encoded[:20])
decoded_pixels = rlc_decode(encoded)
decoded_image = decoded_pixels.reshape(img_array.shape)
Image.fromarray(decoded_image).save("rlc_reconstructed.png")
if np.array_equal(img_array, decoded_image):
    print("\nRLC decoding successful.")
    print("Original and reconstructed images are identical.")
else:
    print("\nError in decoding.")
```

```

original_bits = len(pixels) * 8
compressed_bits = len(encoded) * (8 + 32)
compression_ratio = original_bits / compressed_bits
print("\nOriginal bits:", original_bits)
print("Compressed bits:", compressed_bits)
print("Compression ratio:", compression_ratio)

```

OUTPUT



3. Bitplane Compression

```

from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
image = Image.open("img_2.jpg").convert("L")
img = np.array(image, dtype=np.uint8)
print("Image shape:", img.shape)
print("Minimum pixel value:", img.min())
print("Maximum pixel value:", img.max())
planes = []
for bit in range(8):
    plane = np.zeros(img.shape, dtype=np.uint8)
    for i in range(img.shape[0]):
        for j in range(img.shape[1]):
            shifted = img[i, j] >> bit
            extracted_bit = shifted & 1
            plane[i, j] = extracted_bit
    planes.append(plane)
plt.figure(figsize=(12, 8))
for bit in range(8):
    plt.subplot(2, 4, bit + 1)
    display_plane = planes[bit] * 255
    plt.imshow(display_plane, cmap="gray")
    plt.title("Bit Plane " + str(bit))
    plt.axis("off")
plt.tight_layout()
plt.show()
reconstructed = np.zeros(img.shape, dtype=np.uint8)
for bit in range(8):
    reconstructed += planes[bit] * (2**bit)
if np.array_equal(img, reconstructed):

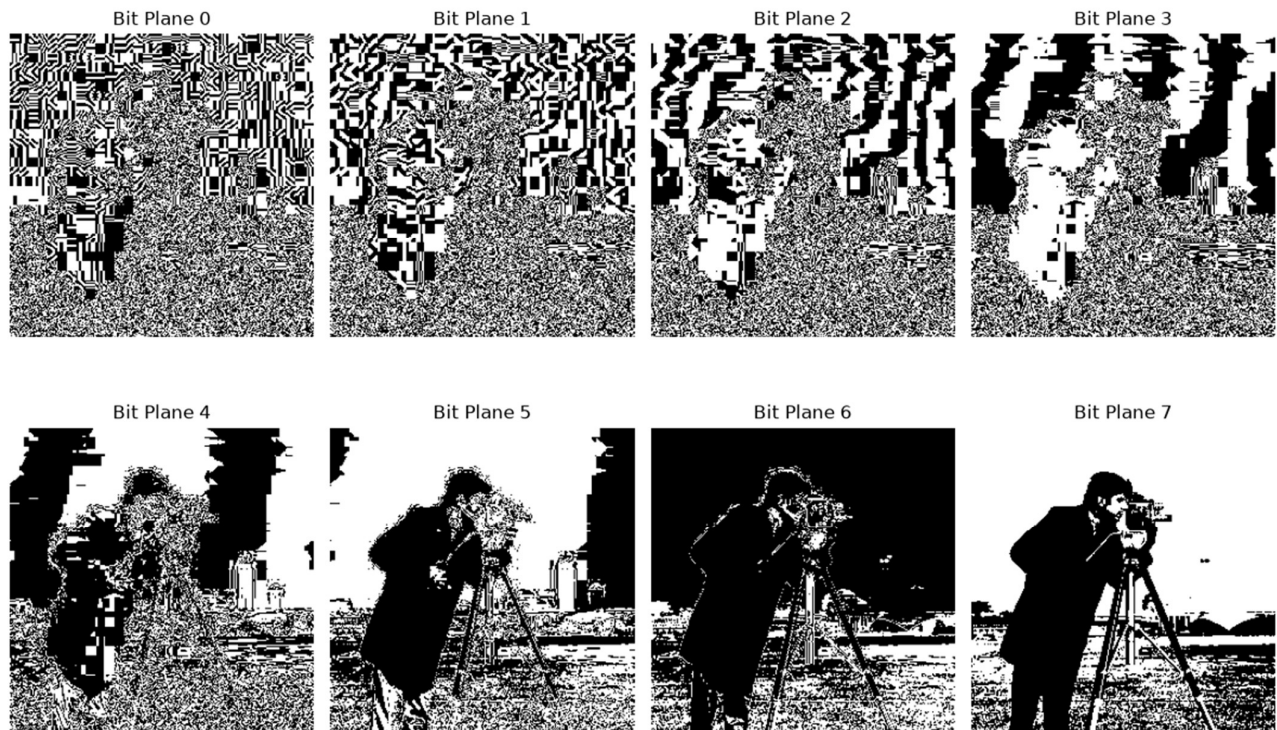
```

```

print("Image reconstructed successfully.")
print("Original and reconstructed images are identical.")
else:
    print("Reconstruction error.")
reconstructed_image = Image.fromarray(reconstructed)
reconstructed_image.save("reconstructed_image.png")
lossy_image = (
    planes[7] * 128 + planes[6] * 64 + planes[5] * 32 + planes[4] * 16
).astype(np.uint8)
Image.fromarray(lossy_image).save("bit_plane_compressed.png")
print("Lossy bit-plane image saved.")

```

OUTPUT



4. DFT

```

from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
image = Image.open("img_2.jpg").convert("L")
img = np.array(image, dtype=np.float64)
img = img[:64, :64]
M, N = img.shape
print("Image size:", M, "x", N)
F = np.zeros((M, N), dtype=complex)
for u in range(M):
    for v in range(N):
        total = 0j
        for x in range(M):
            for y in range(N):

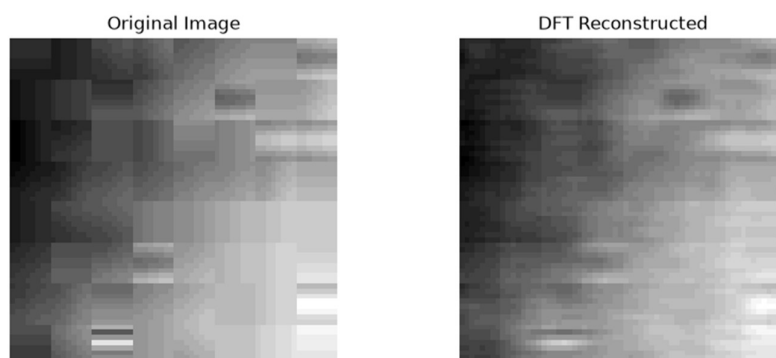
```

```

        angle = -2j * np.pi * ((u * x / M) + (v * y / N))
        total += img[x, y] * np.exp(angle)
    F[u, v] = total
magnitude = np.abs(F)
print("DFT calculated.")
percentage = 10
number_to_keep = int(M * N * percentage / 100)
magnitudes = magnitude.flatten()
sorted_magnitudes = np.sort(magnitudes)
threshold = sorted_magnitudes[-number_to_keep]
compressed_F = np.zeros_like(F)
for u in range(M):
    for v in range(N):
        if abs(F[u, v]) >= threshold:
            compressed_F[u, v] = F[u, v]
reconstructed = np.zeros((M, N), dtype=complex)
for x in range(M):
    for y in range(N):
        total = 0j
        for u in range(M):
            for v in range(N):
                angle = 2j * np.pi * ((u * x / M) + (v * y / N))
                total += compressed_F[u, v] * np.exp(angle)
        reconstructed[x, y] = total / (M * N)
reconstructed = np.real(reconstructed)
reconstructed = np.clip(reconstructed, 0, 255).astype(np.uint8)
Image.fromarray(reconstructed).save("DFT_compressed_reconstructed.png")
print("DFT compression and decompression completed.")
plt.figure(figsize=(10, 4)) plt.subplot(1, 2, 1)
plt.imshow(img, cmap="gray") plt.title("Original Image")
plt.axis("off") plt.subplot(1, 2, 2)
plt.imshow(reconstructed, cmap="gray")
plt.title("DFT Reconstructed") plt.axis("off") plt.show()

```

OUTPUT



5. DCT

```

from PIL import Image
import numpy as np

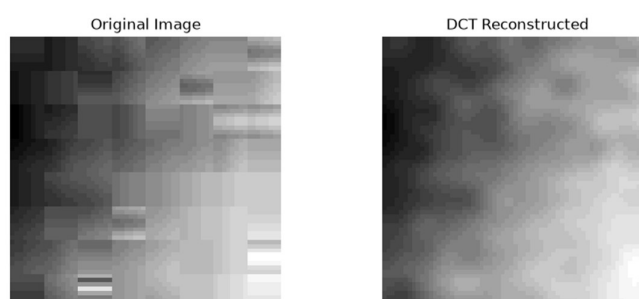
```

```

import matplotlib.pyplot as plt
image = Image.open("img_2.jpg").convert("L")
img = np.array(image, dtype=np.float64)
img = img[:64, :64]
M, N = img.shape
print("Image size:", M, "x", N)
DCT = np.zeros((M, N))
for u in range(M):
    for v in range(N):
        if u == 0: alpha_u = np.sqrt(1 / M)
        else: alpha_u = np.sqrt(2 / M)
        if v == 0: alpha_v = np.sqrt(1 / N)
        else: alpha_v = np.sqrt(2 / N) total = 0
        for x in range(M): for y in range(N):
            cos_x = np.cos(np.pi * (2 * x + 1) * u / (2 * M))
            cos_y = np.cos(np.pi * (2 * y + 1) * v / (2 * N))
            total += img[x, y] * cos_x * cos_y
        DCT[u, v] = alpha_u * alpha_v * total
print("DCT calculated.")keep = 16
compressed_DCT = np.zeros_like(DCT)
compressed_DCT[:keep, :keep] = DCT[:keep, :keep]
reconstructed = np.zeros((M, N))
for x in range(M): for y in range(N):
    total = 0 for u in range(M):
        for v in range(N):if u == 0: alpha_u = np.sqrt(1 / M)
        else: alpha_u = np.sqrt(2 / M)if v == 0: alpha_v = np.sqrt(1 / N)
        else: alpha_v = np.sqrt(2 / N)
        cos_x = np.cos(np.pi * (2 * x + 1) * u / (2 * M))
        cos_y = np.cos(np.pi * (2 * y + 1) * v / (2 * N))
        total += alpha_u*alpha_v*compressedDCT[u,v]*cos_x*cos_y
    reconstructed[x, y] = total
reconstructed = np.clip(reconstructed, 0, 255).astype(np.uint8)
Image.fromarray(reconstructed).save("DCT_compressed_reconstructed.png")
print("DCT compression and decompression completed.")
plt.figure(figsize=(10, 4))plt.subplot(1, 2, 1)
plt.imshow(img, cmap="gray")plt.title("Original Image")
plt.axis("off")plt.subplot(1, 2, 2)
plt.imshow(reconstructed, cmap="gray")
plt.title("DCT Reconstructed")plt.axis("off")plt.show()

```

OUTPUT



6. Plotting DCT DFT RMS Error

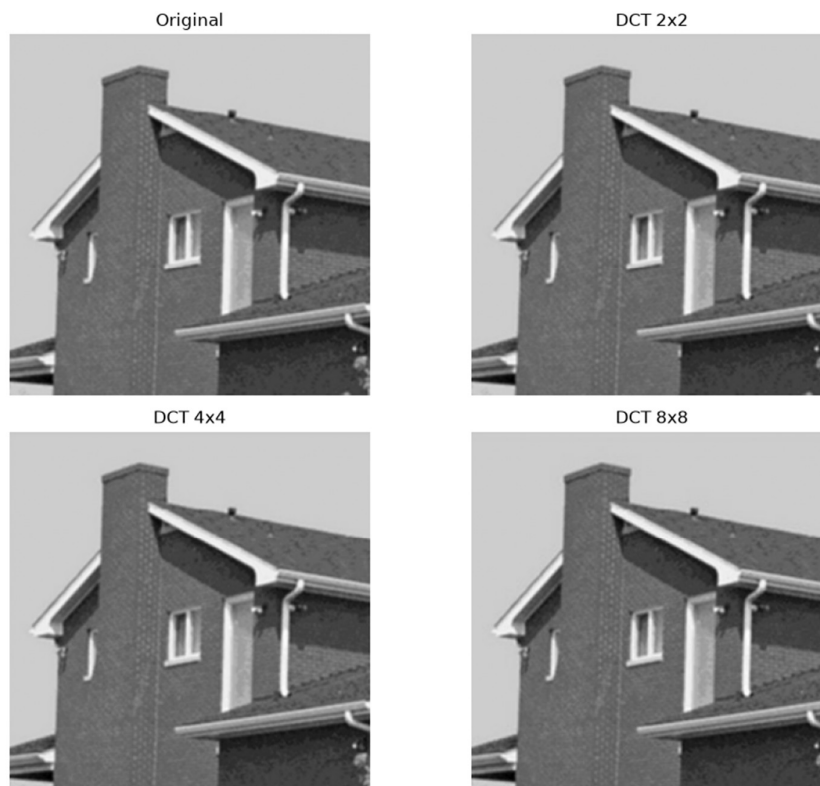
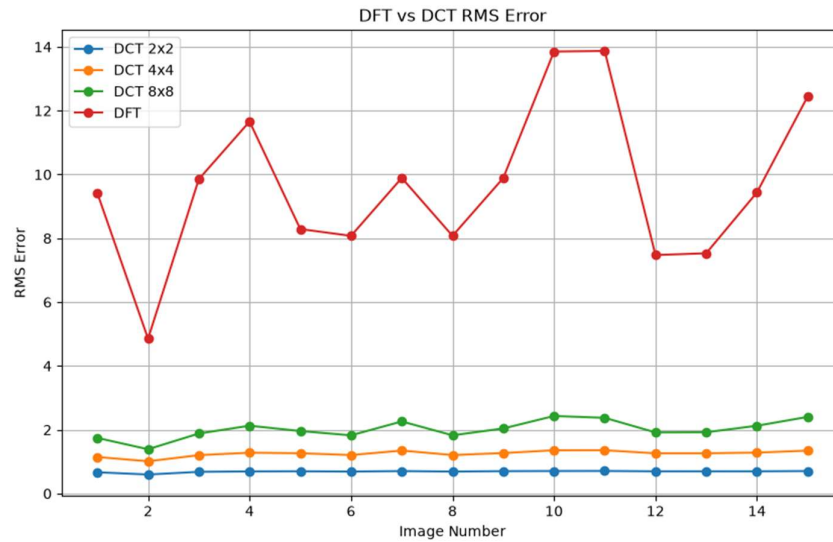
```
import os import matplotlib.pyplot as plt
import numpy as np from PIL import Image
def matrix(n): x = np.arange(n)
    scale = np.full(n, np.sqrt(2 / n))
    scale[0] = np.sqrt(1 / n)
    return scale[:,None]*np.cos(np.pi * (2 * x + 1) * x[:, None] / (2 * n))
def scan(a): n = a.shape[0]
    return [a[i, s - i]
        for s in range(2 * n - 1)
        for i in (range(min(s, n - 1), max(-1, s - n), -1)
            if s % 2 == 0
            else range(max(0, s - n + 1), min(s, n - 1) + 1)
        )]
def unscan(values, n):
    a = np.zeros((n, n), dtype=int) index = 0
    for s in range(2 * n - 1): rows = (
        range(min(s, n - 1), max(-1, s - n), -1)
        if s % 2 == 0 else range(max(0, s - n + 1), min(s, n - 1) + 1))
        for i in rows: a[i, s - i] = values[index]
        index += 1 return a
def rle(values):
    result = [] start = 0
    for end in range(1, len(values) + 1):
        if end == len(values) or values[end] != values[start]:
            result.append((int(values[start]), end - start)) start = end
    return result
def compress(image, size):
    h, w = image.shape h = h - h % size w = w - w % size
    output = np.zeros((h, w)) q = 1 + np.indices((size, size)).sum(axis=0)
    d = matrix(size) encoded = 0
    for i in range(0, h, size):
        for j in range(0, w, size):
            block = image[i : i + size, j : j + size]
            values = np.round(d @ block @ d.T / q).astype(int)
            pairs = rle(scan(values)) encoded += len(pairs)
            values = unscan(
                np.repeat([x for x, _ in pairs], [n for _, n in pairs]), size)
            output[i : i + size, j : j + size] = d.T @ (values * q) @ d
    return (np.clip(output,0,255).astype(np.uint8),image.size/max(encoded,1))
def rms(a, b):
    return np.sqrt(np.mean((a.astype(float) - b.astype(float)) ** 2))
def dft(image):
    transformed = np.fft.fft2(image)
    count = max(1, int(transformed.size * 0.1))
    threshold = np.partition(np.abs(transformed).ravel(), -count)[-count]
    return np.clip(np.real(
```

```

        np.fft.ifft2(np.where(np.abs(transformed) >= threshold,
transformed, 0)))0,255,).astype(np.uint8)
def main():
    folder = os.path.join(os.path.dirname(os.path.abspath(__file__)), "img")
    if not os.path.exists(folder):
        raise FileNotFoundError(f"Image folder not found: {folder}")
    names = sorted(n for n in os.listdir(folder) if
n.lower().endswith(".png"))[:15]
    if not names:
        raise FileNotFoundError(f"No PNG images found in: {folder}")
    images = []
    for n in names:
        path = os.path.join(folder, n)
        try:
            image = Image.open(path)
            image.load()
            image = image.convert("L")
            image = image.resize((256, 256))
            images.append(np.asarray(image))
        except Exception:
            print("Skipped:", n)
    if not images:
        raise FileNotFoundError("No valid PNG images found.")
    print("Images used:")
    for i, n in enumerate(names[: len(images)], 1):
        print(i, n)
    sizes = (2, 4, 8)
    errors = {f"DCT {size}x{size}": [] for size in sizes}
    errors["DFT"] = []
    for image in images:
        for size in sizes:
            compressed, _ = compress(image, size)
            errors[f"DCT {size}x{size}"].append(rms(image, compressed))
        errors["DFT"].append(rms(image, dft(image)))
    x = np.arange(1, len(images) + 1)
    plt.figure(figsize=(10, 6))
    for label, values in errors.items():
        plt.plot(x, values, "o-", label=label)
    plt.xlabel("Image Number")
    plt.ylabel("RMS Error")
    plt.title("DFT vs DCT RMS Error")
    plt.legend()
    plt.grid()
    plt.show()
    original = images[0]
    dct2 = compress(original, 2)[0]
    dct4 = compress(original, 4)[0]
    dct8 = compress(original, 8)[0]
    results = [
        ("Original", original),
        ("DCT 2x2", dct2),
        ("DCT 4x4", dct4),
        ("DCT 8x8", dct8),
    ]
    plt.figure(figsize=(10, 8))
    for i, (title, result) in enumerate(results, 1):
        plt.subplot(2, 2, i)
        plt.imshow(result, cmap="gray")
        plt.title(title)
        plt.axis("off")
    plt.tight_layout()
    plt.show()
if __name__ == "__main__":
    main()

```

OUTPUT



7. JPEG Compression

```
import heapq from collections import Counter
import numpy as np from PIL import Image
Q = np.array([
    [16, 11, 10, 16, 24, 40, 51, 61],
    [12, 12, 14, 19, 26, 58, 60, 55],
    [14, 13, 16, 24, 40, 57, 69, 56],
    [14, 17, 22, 29, 51, 87, 80, 62],
    [18, 22, 37, 56, 68, 109, 103, 77],
    [24, 35, 55, 64, 81, 104, 113, 92],
    [49, 64, 78, 87, 103, 121, 120, 101],
```

```

[72, 92, 95, 98, 112, 100, 103, 99],])
n = 8 x = np.arange(n) s = np.full(n, np.sqrt(2 / n))
s[0] = np.sqrt(1 / n)
C = s[:, None] * np.cos(np.pi * (2 * x + 1) * x[:, None] / (2 * n))
def zigzag(a): return [
    a[i, s - i] for s in range(15) for i in (
        range(min(s, 7), max(-1, s - 8), -1) if s % 2 == 0
        else range(max(0, s - 7), min(s, 7) + 1)]
def unzigzag(values): a = np.zeros((8, 8), dtype=int) k = 0
for s in range(15): rows = (
    range(min(s, 7), max(-1, s - 8), -1) if s % 2 == 0
    else range(max(0, s - 7), min(s, 7) + 1))
for i in rows: a[i, s - i] = values[k] k += 1 return a
def rle(values): result = [] start = 0
for end in range(1, len(values) + 1):
    if end == len(values) or values[end] != values[start]:
        result.append((int(values[start]), end - start)) start = end
return result
def huffman(values): counts = Counter(values)
if len(counts) == 1:
    value = next(iter(counts))
    return "0" * counts[value], {value: "0"}
heap = [[count, [value, ""]] for value, count in counts.items()]
heapq.heapify(heap)
while len(heap) > 1:
    left = heapq.heappop(heap) right = heapq.heappop(heap)
    for item in left[1:]:
        item[1] = "0" + item[1]
    for item in right[1:]:
        item[1] = "1" + item[1]
    heapq.heappush(heap, [left[0] + right[0], *left[1:], *right[1:]])
codes=dict(heap[0][1:]) bits = "".join(codes[value] for value in values)
return bits, codes
def block_encode(block):
    dct = np.round(C @ (block - 128) @ C.T / Q).astype(int)
    return rle(zigzag(dct))
def block_decode(data):
    values = np.repeat(
        [value for value, count in data], [count for value, count in data])
    values = values[:64]
    if len(values) < 64: values = np.pad(values, (0, 64 - len(values)))
    return np.clip(C.T @ (unzigzag(values) * Q) @ C + 128, 0, 255)
def compress(image): image = np.asarray(image, dtype=float)
h, w = image.shape
ph = (h + 7) // 8 * 8 pw = (w + 7) // 8 * 8
padded = np.zeros((ph, pw)) padded[:h, :w] = image
blocks = [] for i in range(0, ph, 8):
    for j in range(0, pw, 8): block = padded[i : i + 8, j : j + 8]
    blocks.append(block_encode(block))

```

```

    data = [pair for block in blocks for pair in block]
    bits, codes = huffman(data)
    return bits, codes, (ph, pw), blocks
def decompress(blocks, shape, padded_shape):
    output = np.zeros(padded_shape)
    k = 0
    for i in range(0, padded_shape[0], 8):
        for j in range(0, padded_shape[1], 8):
            output[i : i + 8, j : j + 8] = block_decode(blocks[k])
            k += 1
    return output[: shape[0], : shape[1]]
image = Image.open("img_2.jpg").convert("L")
original = np.asarray(image)
bits, codes, padded_shape, blocks = compress(original)
reconstructed = decompress(blocks, original.shape, padded_shape)
Image.fromarray(reconstructed.astype(np.uint8)).save("jpeg_decompressed.png")
rms = np.sqrt(np.mean((original.astype(float) - reconstructed) ** 2))
print("Original:", original.shape)
print("Compressed bits:", len(bits))
print("RMS error:", rms)

```

OUTPUT

